Sweet And Savory

AlgoExpert · Medium · Arrays

The problem

- Dishes have negative (sweet) or positive (savory) flavour.
- Pick one sweet + one savory whose sum is closest to target without exceeding it; return [sweet, savory] or [0, 0].

Intuition

- Sort sweet and savory dishes, then use two pointers to approach the target from below.

Brute force vs optimal

- Brute force: all sweet × savory pairs → $O(n^2)$
- Sort each side + two pointers → $O(n \log n)$
- Never exceed the target

Approach

- 1. Split by sign and sort
- 2. Pointer on the mildest sweet and the smallest savory
- 3. If the sum fits, record it and try a bigger savory
- 4. Otherwise try a sweeter dish

Worked example

sweet (mildest first)



savory (smallest first)




```
sum = -2 vs target 8: fits → record, try bigger savory
best = [-3, 1]
```

-2 ≤ 8? fits → record, try bigger savory

Worked example

sweet (mildest first)



savory (smallest first)




```
sum = 4 vs target 8: fits → record, try bigger savory
best = [-3, 7]
```

4 ≤ 8? fits → record, try bigger savory

Java solution

```java
public class SweetAndSavory {
    public static int[] sweetAndSavory(int[] dishes, int target) {
        List<Integer> sweet = new ArrayList<>(), savory = new ArrayList<>();
        for (int d : dishes) if (d < 0) sweet.add(d); else if (d > 0) savory.add(d);
        sweet.sort(Collections.reverseOrder());
        Collections.sort(savory);
        int[] best = {0, 0};
        long bestDiff = Long.MAX_VALUE;
        int i = 0, j = 0;
        while (i < sweet.size() && j < savory.size()) {
            long sum = (long) sweet.get(i) + savory.get(j);
            if (sum <= target) {
                if (target - sum < bestDiff) { bestDiff = target - sum; best = new int[]{sweet.get(i), savory.get(j)}; }
                j++;
            } else i++;
        }
        return best;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- No valid pair → [0, 0]
- Zero-flavour dishes are ignored
- Sum must not exceed target

Interview tips & follow-ups

- Explain pointer movement carefully
- Follow-up: allow exceeding target by closest absolute diff

Recap

- Sort sweet and savory dishes, then use two pointers to approach the target from below.
- Complexity: O(n log n)